

CREACIÓN DEL PROYECTO FRONTEND



Implementación del servicio `PersonaService` en Angular

Introducción

En esta lección se construirá un servicio en Angular llamado `PersonaService`, el cual será responsable de administrar un arreglo local de personas en la aplicación. Este servicio servirá de puente entre la interfaz del usuario y el servicio web del backend, permitiendo agregar, buscar, modificar y eliminar personas, manteniendo sincronizados los datos con el backend de Java.

Paso 1: Crear la clase `PersonaService`

Descripción:

Se crea un nuevo servicio en Angular que manejará las operaciones sobre un arreglo local de personas. Esta clase interactuará con `DataService` para enviar y recibir datos del backend.

Ruta del archivo:

```
src/app/persona-service.ts
```

Comando utilizado:

```
ng g class PersonaService --skip-tests
```

Paso 2: Definir e importar dependencias

Descripción:

Importamos las clases necesarias para construir el servicio y habilitamos la inyección de dependencias con el decorador `@Injectable`.

Código:

```
import { Injectable } from '@angular/core';
import { Persona } from './persona.model';
import { DataService } from './data-service';
```

```
@Injectable()
export class PersonaService {
```

Explicación:

- Se importa `Persona` como modelo de datos.
 - Se importa `DataService` que nos permite conectarnos al backend.
 - Se declara la clase como un servicio con `@Injectable`.
-

Paso 3: Declarar el arreglo de personas

Descripción:

Declaramos un arreglo de personas vacío como propiedad de la clase.

 Código:

```
personas: Persona[] = [];
```

 Explicación:

Este arreglo servirá como almacenamiento temporal de los datos obtenidos del backend.

 Paso 4: Constructor e inyección del `DataService` **Descripción:**

Se crea un constructor con inyección del servicio `DataService`.

 Código:

```
constructor(private dataService: DataService) {}
```

 Explicación:

Con esto, `PersonaService` puede usar los métodos de `DataService` para comunicarse con el backend.

 Paso 5: Método `setPersonas()` **Descripción:**

Se crea un método para inicializar el arreglo local una vez obtenidos los datos desde el servidor.

 Código:

```
setPersonas(personas: Persona[]) {  
    this.personas = personas;  
}
```

 Explicación:

Este método será útil al recuperar datos de forma asíncrona y luego actualizar el arreglo local.

+ Paso 6: Método `agregarPersona()`

Descripción:

Se implementa el método para agregar una nueva persona al backend y, si es exitoso, al arreglo local.

Código:

```
agregarPersona(persona: Persona) {  
  console.log('persona a agregar:' + persona.nombre);  
  this.dataService.agregarPersona(persona)  
    .subscribe({  
    next: (persona: Persona) => {  
      console.log('se agrega al arreglo la persona recién insertada suscriber:' + persona.idPersona);  
      this.personas.push(persona);  
    },  
    error: (error) => {  
      console.error('Error al agregar persona:', error);  
    }  
  });  
}
```

Explicación:

- Se imprime el nombre antes de enviarlo.
- Al recibir la respuesta (con ID ya generado), se inserta al arreglo local.
- También se manejan errores.

Paso 7: Método `encontrarPersona()`

Descripción:

Permite buscar una persona por su `idPersona` en el arreglo local.

Código:

```
encontrarPersona(id: number){  
  const persona = this.personas.find(p => p.idPersona == id);  
  console.log('persona encontrada:' + persona?.idPersona + ' ' + persona?.nombre);  
}
```

```
    return persona;
}
```

Explicación:

Se usa `.find()` para localizar la persona y se imprime su información si se encuentra.

Paso 8: Método `modificarPersona()`

Descripción:

Llama al backend para modificar la persona. No se modifica aún el arreglo local, solo se envía al servidor.

Código:

```
modificarPersona(id: number, persona: Persona) {
  console.log('persona a modificar:' + persona.idPersona);
  this.dataService.modificarPersona(id, persona);
}
```

Explicación:

Simplemente se envía la modificación al backend y se imprime un mensaje.

Paso 9: Método `eliminarPersona()`

Descripción:

Elimina una persona del arreglo local y también del backend.

Código:

```
eliminarPersona(id: number) {
  console.log('eliminar persona con id:' + id);
  const index = this.personas.findIndex(p => p.idPersona == id);
  this.personas.splice(index, 1);
  this.dataService.eliminarPersona(id);
}
```

Explicación:

- Busca el índice del objeto con `.findIndex()`.
- Lo elimina localmente con `.splice()`.
- Luego se envía la solicitud de eliminación al backend.

Sección para el código final completo

Aquí puedes colocar el **código final completo** del archivo trabajado:

 `src/app/persona-service.ts`

```
import { Injectable } from '@angular/core';
import { Persona } from '../persona.model';
import { DataService } from '../data-service';

@Injectable()
export class PersonaService {

  personas: Persona[] = [];

  constructor(private dataService: DataService) {}

  setPersonas(personas: Persona[]) {
    this.personas = personas;
  }

  agregarPersona(persona: Persona) {
    console.log('persona a agregar:' + persona.nombre);
    this.dataService.agregarPersona(persona)
      .subscribe({
        next: (persona: Persona) => {
          console.log('se agrega al arreglo la persona recién insertada suscribir:' + persona.idPersona);
          this.personas.push(persona);
        },
        error: (error) => {
          console.error('Error al agregar persona:', error);
        }
      });
  }

  encontrarPersona(id: number){
```

```
const persona = this.personas.find(p => p.idPersona == id);
console.log('persona encontrada:' + persona?.idPersona + ' ' + persona?.nombre);
return persona;
}

modificarPersona(id: number, persona: Persona) {
  console.log('persona a modificar:' + persona.idPersona);
  this.dataService.modificarPersona(id, persona);
}

eliminarPersona(id: number) {
  console.log('eliminar persona con id:' + id);
  const index = this.personas.findIndex(p => p.idPersona == id);
  this.personas.splice(index, 1);
  this.dataService.eliminarPersona(id);
}
}
```



Conclusión

En esta guía aprendiste cómo crear una clase `PersonaService` que actúa como administrador del arreglo de personas en Angular. Este servicio permite interactuar fácilmente con el backend utilizando `DataService` para mantener actualizados los datos y reflejar los cambios en la interfaz del usuario. ¡Una pieza clave en la arquitectura moderna de aplicaciones web! 🤖

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)